Accelerating Bayesian Optimization for Protein Design via Evolutionary Acquisition Optimization

Pranshu Panda
University of Minnesota Twin Cities
panda068@umn.edu

Drew Gjerstad
University of Minnesota Twin Cities
gjers043@umn.edu

Code: https://github.com/pranshupanda/accelerating-bo

1 Introduction

Protein design plays a central role in modern drug discovery and therapeutic development, yet its progress remains limited by the pace of wet-lab experimentation. Each evaluation of a candidate sequence requires synthesis, folding, and testing, creating a huge bottleneck in the experimentation and evaluation cycle. These constraints require us to consider computational methods capable of accelerating discovery by reducing the number of costly experimental evaluations. Bayesian optimization offers a powerful framework for such settings by using uncertainty-aware surrogate models and acquisition functions to guide the search for optimal designs.

Although several state-of-the-art methods for optimizing black-box and expensive-to-evaluate objective functions in continuous high-dimensional domains exist, they typically rely on gradient-based techniques to efficiently navigate the large search space. Unfortunately, gradients are undefined in discrete domains such as those found in protein design and thus renders our objective outside the scope of these methods. More importantly, one desideratum for our optimizer is its capability to make large jumps in the search space and consequently progress efficiently. We propose to leverage a new class of evolutionary algorithms capable of large jumps in the search space to make progress efficiently while being equally adept at local hill climbing compared to existing optimization methods [2] [4] [3].

Our goal for sample-efficient protein design will place the Bayesian optimization framework at the core of our work and is also one of the pillars of this course. Furthermore, we will use advancements in evolutionary algorithms in tandem with acquisition functions and their optimizers to efficiently optimize black-box, expensive-to-evaluate objectives with discrete inputs which quantify the utility of candidate proteins [6]. Thus, our proposed work directly incorporates content from this course.

2 Problem Statement

We consider the problem of sample-efficient optimization in a high-dimensional discrete search space motivated by protein design under expensive evaluation constraints.

Let $\mathcal{X} = \mathcal{A}^d$ denote a discrete sequence space, where each $x \in \mathcal{X}$ is a sequence of length d over a finite alphabet $\mathcal{A}$. In the benchmark setting, we additionally consider the Low Autocorrelation Binary Sequences (LABS) problem, where $\mathcal{X} = \{0, 1\}^d$ [6]. The resulting search space is combinatorial and grows exponentially with d.

We assume an unknown black-box objective function:

$$f : \mathcal{X} \to \mathbb{R},$$

which assigns a scalar merit score to each sequence. In the protein design setting, this score represents properties such as stability, binding affinity, or functional feasibility. In the LABS setting, it corresponds to a synthetic energy-like objective.

The optimization problem is:

$$\max_{x \in \mathcal{X}} f(x),$$

where evaluations of f are expensive, reflecting costly experiments or high-fidelity simulations. As a result, the number of function evaluations is strictly limited to a budget T, yielding a dataset $\mathcal{D} = \{(x_i, f(x_i))\}_{i=1}^{n}$ with $n \leq T$.

The objective landscape is highly non-convex, rugged, and lacks gradient information due to the discrete nature of the domain. This makes exhaustive search infeasible and renders the problem one of sample-efficient global optimization under strict evaluation budgets. Unlike standard Bayesian optimization settings typically covered in lecture, which assume continuous domains $\mathcal{X} \subset \mathbb{R}^d$ and rely on smoothness to enable efficient optimization in continuous space, our setting is discrete and combinatorial, lacking meaningful notions of gradients or local geometry.

Goal. The goal is to identify high-merit sequences in $\mathcal{X}$ using as few evaluations of f as possible under the constraint $n \leq T$.

3 Approach

Our approach builds on the Bayesian optimization (BO) framework and adapts it to discrete, high-dimensional sequence spaces by integrating evolutionary search strategies for acquisition optimization.

3.1 Bayesian Optimization Framework

We model the unknown objective function $f : \mathcal{X} \to \mathbb{R}$ using a probabilistic surrogate, typically a Gaussian Process (GP) which is trained on limited observed data $\mathcal{D} = \{(x_i, f(x_i))\}_{i=1}^{n}$. The surrogate defines a posterior distribution $p(f \mid \mathcal{D})$, which provides both predictive mean estimates and uncertainty over candidate sequences.

The model is composed of a covariance or kernel function $k(x, x')$, which defines the covariance or similarity between any two points in the input space and a mean function which defines the expected value of the function for some input. In our setting, where $\mathcal{X}$ is a discrete and combinatorial space, the choice of kernel is critical, as the notion of distance breaks down in high dimensions.

Based on this surrogate, which we can now query frequently instead of the computationally expensive black-box objective, we define an acquisition function:

$$a(x; \mathcal{D}),$$

which measures the utility of evaluating a candidate sequence by trading off exploration and exploitation. At each iteration, Bayesian optimization selects the next query point via:

$$x_{t+1} = \arg\max_{x \in \mathcal{X}} a(x; \mathcal{D}_t).$$

Because $\mathcal{X}$ is combinatorial, the acquisition maximization problem is itself a non-trivial black-box optimization task. The absence of continuous structure or gradients prevents the use of standard gradient-based optimizers, requiring specialized discrete search procedures to approximately solve:

$$\max_{x \in \mathcal{X}} a(x; \mathcal{D}).$$

3.2 Fast Contiguous Hypermutations (CHM)

We use the Fast Contiguous Somatic Hypermutation (CHM) operator proposed by Corus et al. (2025) as the mutation operator within the B-Cell Algorithm (BCA) used for acquisition maximization [2].

Let $x \in \mathcal{X} = \{0,1\}^d$. A CHM mutation selects a starting index $a \in \{1,\dots,d\}$ uniformly at random and flips a contiguous segment of bits. For each position $i \in \{1,\dots,d\}$ along the scan, the bit is flipped with probability $r \in [0,1]$:

$$x[(a+i) \bmod d] = 1 - x[(a+i) \bmod d].$$

In the standard CHM operator, the length of the mutated contiguous region is randomly determined, and each bit within the region is flipped independently with probability r (typically $r = 1$ in prior analyses)[3]. This allows CHM to perform large contiguous mutations, which is beneficial for escaping local optima in rugged landscapes. However, CHM is known to be inefficient in hillclimbing phases, since the same mechanism is used regardless of whether small or large mutations are required.

The Fast CHM operator of Corus et al. (2025) modifies this behavior by introducing a position-dependent evaluation probability p_i after each bit flip:

$$p_i = \max\left((\min\{i, d-i+1\})^{-\beta}, \frac{1}{d}\right), \quad \beta > 1.$$

Unlike CHM, Fast CHM evaluates intermediate solutions during the mutation process according to p_i, enabling efficient evaluation of small local changes while preserving the ability to perform large contiguous mutations. This leads to improved performance during hillclimbing phases while maintaining the exploratory behavior of CHM.

Comparison. CHM performs a full contiguous mutation before evaluation, which can be inefficient when only small improvements are needed. Fast CHM instead introduces intermediate evaluations with higher probability near small mutation lengths, making it asymptotically as efficient as standard bit mutation (SBM) in hillclimbing regimes[4], while retaining CHM's ability to exploit large contiguous jumps [2].

3.3 BCA for Acquisition Maximization

We embed CHM/Fast CHM within the B-Cell Algorithm (BCA), which serves as the optimizer for the acquisition function in Bayesian optimization.

At each BO iteration t, given acquisition function $a(x; \mathcal{D}_t)$, BCA applies the Fast CHM operator to the sequence until we converge to the best candidate while querying the surrogate for evaluation. This candidate sequence is evaluated by the true function f and added to the training dataset.

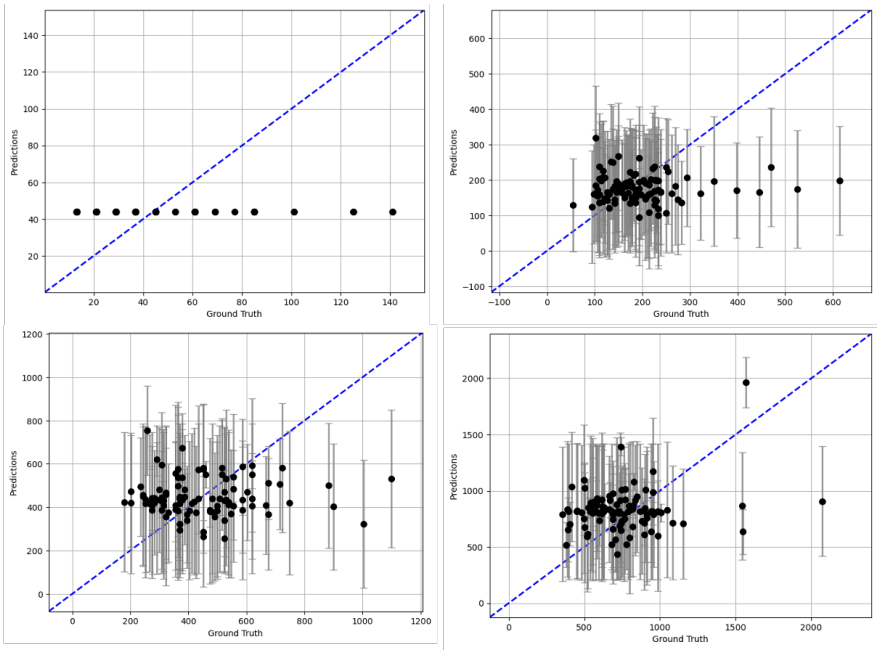
It should be noted that this procedure was parallelized to optimize run time, but this diverges from the nature of this algorithm. A thorough discussion of this will be conducted in the Analysis section.

Bayesian Optimization with BCA
$Dataset \mathcal{D}_0$ initialized
for $t = 1, \dots, T$
  Surrogate Model $\mathcal{M}_t$ = fit on $\mathcal{D}_{t-1}$
  Acquisition Function $a_t(x)$ defined, $x = x_0$
  while not converged
    x' = Fast-CHM(x)
    if $\alpha_t(x') > \alpha_t(x)$, **then** $x = x'$
  end while
  $x_t = x, \quad y_t = f(x_t)$
  $\mathcal{D}_t = \mathcal{D}_{t-1} \cup \{(x_t, y_t)\}$
end for

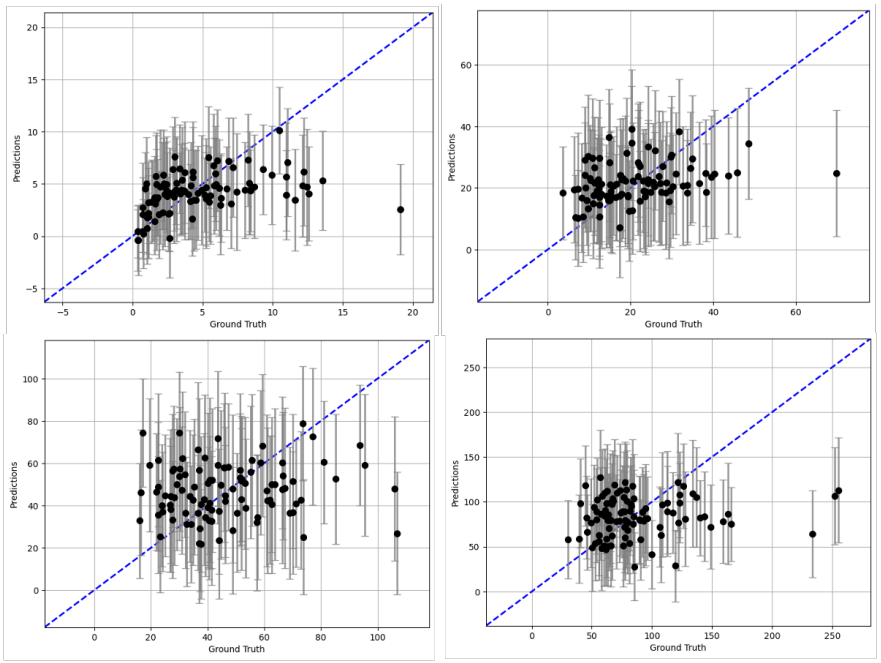
4 Results and Analysis

4.1 Kernel Selection

The Radial Basis Function (RBF) kernel and the categorical kernel provided by BoTorch were compared based on their performance in predicting the output of the LABS Benchmark. [1] [6]



(a)



(b)

Figure 1: Model evaluation with GP for 10, 20, 30, and 40 dimensions on LABS with 100 training and test samples for (a) Categorical Kernel and (b) Radial Basis Function (RBF) Kernel

A qualitative evaluation of the kernels in 1 shows that though the Categorical Kernel struggles to capture the structure of the 10 dimensional LABS problem, it has much better predictions than RBF Kernel on higher dimensional problems due to the limited spread of predictions. Since the domain of protein design is concerned with high dimensional inputs, the Categorical Kernel is preferred. As discussed in the approach, we expected the notion of distance to break down, and it is verified by the performance of the RBF Kernel which hinges on Euclidean distance computation. The Mean Squared Error (MSE) was not completed for these experiments, thus the result is not a quantitative evaluation.

4.2 Direct Optimization with Evolutionary Algorithms

BCA was compared with other evolutionary approaches, such as the Firefly Algorithm and Particle Swarm Optimization (PSO) [7] [5]. For each method, we measure best-found objective values denoted by BEST *dimensionality* for brevity in 1 and 2. We include the algorithm Optimal to denote global maximums for each problem.

Evolutionary Algorithm	BEST10	BEST20	BEST30	BEST40	BEST50	BEST60
Optimal	3.846	7.692	7.627	7.407	8.170	8.257
BCA	3.846	5.387	4.926	4.084	3.892	3.805
Firefly	3.672	7.191	4.656	3.859	3.503	3.150
PSO	3.846	6.438	4.634	3.982	3.427	3.158

Table 1: Performance of Evolutionary Algorithms on maximizing LABS, 10 independent trials consisting of 10000 iterations each [6].

We notice that BCA gets outperformed by Firefly and PSO in low dimensions. We hypothesize this is due to the population-based nature of these algorithms which allow leveraging parallelism to efficiently solve optimization problems. In high dimensional space, we hypothesize that the increasingly sparse search space benefits BCA's search dynamics. It consistently outperforms Firefly and PSO in 30, 40, 50 and 60 dimensions.

4.3 Bayesian Optimization (BO) using Evolutionary Acquisition Optimizers

Within the BO framework, we now leverage BCA, PSO and Firefly as acquisition function optimizers. We also compared each of these methods to Discrete Local Search which is offered by BoTorch and is tailor-made for discrete search spaces [1]. It serves as a baseline to evaluate performance due to known characteristics and provide a reference point for improvement.

Acquisition Optimizer	BEST10	BEST20	BEST30	BEST40	BEST50
Optimal	3.846	7.692	7.627	7.407	8.170
BCA	3.636	3.757	3.612	3.624	2.976
Firefly	3.627	3.313	2.629	2.387	2.058
PSO	3.650	3.972	3.416	2.810	2.213
LocalSearch	3.638	3.825	3.647	3.499	2.945

Table 2: Performance of Bayesian Optimization(BO) using Evolutionary Acquisition Function Optimizers on maximizing LABS, 10 independent trials consisting of 100 BO iterations each [6].

The relatively similar performance between BCA and Local Search suggests that BoTorch's discrete Local Search implementation is already a strong optimizer for this problem setting. Although BCA achieves slight improvements in the 40 and 50 dimensional cases, the magnitude of these gains is small and does not provide strong evidence that BCA consistently outperforms the baseline. Both methods significantly outperform PSO and Firefly, but it is important to notice no algorithm converges to the optimal solution with increasing degradation in solution quality with respect to dimensionality. Additional trials would be required to determine whether the differences between BCA and Local Search are statistically significant.

4.4 Insights and Limitations

Performance was evaluated using the best objective value found during optimization. Since LABS is a maximization problem, higher values indicate better solutions. For the kernel comparison experiments, predictive performance was assessed qualitatively through visually analzying predictive and ground truth output plots. All experiments were repeated across 10 independent random trials to reduce the impact of stochastic fluctuations, however we note the number of runs should have been scaled up to further reduce it's impact. The consistency of BCA's advantage in direct optimizations of LABS dimensions 30–60 suggests the trend is unlikely to be random, though formal statistical tests such as a paired t-test were not conducted. The same LABS benchmark objective function and dimensional settings were used across all methods to ensure fair comparison. Each evolutionary optimization run used 10,000 iterations, while Bayesian Optimization experiments used 100 BO iterations.

5 Conclusion

We learned that outperforming strong baselines such as BoTorch's discrete local search is non-trivial and requires careful design of both the surrogate model and the acquisition optimization strategy. We also learned that optimization of a method is a challenging systems task and pitfalls such as divergence from algorithm logic is possible and should be avoided at all costs. One can consider the evaluation skewed by such a divergence, which was one of the challenges we encountered through this project. Finally, one of the challenges we faced were suboptimal implementations of functions that could have been imported using the correct libraries, requiring more thorough research of available resources. Such implementations led to unfavorable results for the 60 dimensional case of LABS in the BO setting where it timed out on MSI after taking in excess of 24 hours[6]. We believe that fundamentally our project is representative of what was proposed, but on an implementation level there has been divergence that affects our results. Unfortunately, we were not able to extend our work to the Machine Learning Life Science HDBO benchmark which would have been more representative of our protein design search space.

6 Contributions

Pranshu was responsible for the development of all of the code in the repository, logging experiments, writing the Problem Statement, Approach, Results and Analysis and Conclusion sections of the final report. Drew was responsible for the Introduction section and improving code structure in the repository and proofreading.

7 Generative AI Statement

Generative AI was used in an assistive capacity through the course of this report (mainly to rephrase content), resolve some LaTeX issues, debug and optimize handwritten code and suggest changes to code structure for clarity.

References

[1] Maximilian Balandat, Brian Karrer, Daniel R. Jiang, Samuel Daulton, Benjamin Letham, Andrew Gordon Wilson, and Eytan Bakshy. BoTorch: A Framework for Efficient Monte-Carlo Bayesian Optimization. In *Advances in Neural Information Processing Systems 33*, 2020. URL http://arxiv.org/abs/1910.06403.

[2] Dogan Corus, Pietro S. Oliveto, and Donya Yazdani. Fast contiguous somatic hypermutations for single-objective optimisation and multi-objective optimisation via decomposition. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39:26922–26930, 2025. doi: 10.1609/aaai.v39i25.34897. URL https://doi.org/10.1609/aaai.v39i25.34897.

[3] Thomas Jansen and Christine Zarges. Analyzing different variants of immune inspired somatic contiguous hypermutations. *Theoretical Computer Science*, 412(6):517–533, 2011. ISSN 0304-3975. doi: https://doi.org/10.1016/j.tcs.2010.09.027. URL https://www.sciencedirect.com/science/article/pii/S0304397510005062. Theoretical Aspects of Artificial Immune Systems.

[4] Johnny Kelsey and Jon Timmis. Immune inspired somatic contiguous hypermutation for function optimisation. In *Genetic and Evolutionary Computation — GECCO 2003*, pages 207–218, Berlin, Heidelberg, 2003. Springer Berlin Heidelberg.

[5] J. Kennedy and R. Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95 - International Conference on Neural Networks*, volume 4, pages 1942–1948 vol.4, 1995. doi: 10.1109/ICNN.1995.488968.

[6] Tom Packebusch and Stephan Mertens. Low autocorrelation binary sequences. *Journal of Physics A: Mathematical and Theoretical*, 49(16):165001, March 2016. ISSN 1751-8121. doi: 10.1088/1751-8113/49/16/165001. URL http://dx.doi.org/10.1088/1751-8113/49/16/165001.

[7] Xin-She Yang. Firefly algorithms for multimodal optimization, 2010. URL https://arxiv.org/abs/1003.1466.